

PRACTICUM WORKSHEET: THE AMDAHL REALITY GAP

Empirical Speedup Fitting, Micro-Architectural Bottlenecks, and Loop Scheduling in OpenMP

Student Full Name: <u>Adel Kunuspekova</u>	Student ID Number: <u>230103231</u>
Institutional Email / Git URL: <u></u>	Practicum Date & Time (Slot): <u>24.09.2026</u>
Laptop Model & OS (Linux/macOS/Win): <u>ASUS Vivobook / Windows 11</u>	CPU Model & Generation: <u>11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz</u>
Physical Cores (P/E breakdown if hybrid): <u>4</u>	Logical SMT Threads (omp_get_max_threads): <u>8</u>
Target Workload Size N (Derived via Formula): <u>13231000</u>	L1 / L2 Cache Line Size (Typically 64 bytes): <u>64</u>

LAB INTEGRITY & ANTI-PLAGIARISM PROTOCOL: STRICT INDIVIDUAL EXECUTION & ZERO-TOLERANCE INTEGRITY NOTICE: This laboratory practicum evaluates real physical execution on your personal laptop hardware. Generative AI tools (ChatGPT, Copilot, Claude) can produce synthetic OpenMP syntax, but they CANNOT execute native code on your silicon, cannot generate valid microsecond wall-clock timings reflecting your CPU's hardware architecture, and cannot produce the exact checksum tied to your unique Student ID seed. All logs, hardware dumps, and execution timestamps will be audited automatically. Fabricated or simulated data results in immediate grade revocation (0/100). Do not work in pairs.

1. Overview & Theoretical Framework

Amdahl's Law models the theoretical upper bound of parallel speedup $S(k)$ as a function of the parallelizable fraction p and the number of execution units k :

AMDAHL'S LAW: $S(k) = 1 / [(1 - p) + (p / k)]$
Where: $S(k)$ = Speedup with k worker threads, p = Fraction of execution time strictly parallelizable ($0 \leq p \leq 1$), and $(1 - p)$ = Strictly sequential component.

In textbooks, Amdahl's Law assumes that adding execution threads incurs zero communication latency, perfect memory bandwidth scaling, and uniform instruction latency. On real physical hardware, however, programs encounter the Amdahl Reality Gap: performance degrades below theoretical bounds due to:

- **1. Sequential Overhead:** Sequential fraction bottlenecks $(1 - p)$ such as thread spawning, barrier waits, and reduction synchronization.
- **2. False Sharing:** Multiple cores thrashing the same 64-byte L1/L2 cache line via the MESI cache coherence protocol.
- **3. Workload Imbalance:** Adjacent iterations consuming drastically different execution cycles, causing worker threads to idle at barriers.
- **4. Memory & Execution Bandwidth Saturation:** Memory controller saturation and Simultaneous Multithreading (SMT) hyper-thread pipeline contention.

Laboratory Practicum Roadmap (180 Minutes Minimum Execution)

- Phase 1 (20 mins): System architecture probing, hardware dump, and unique seed derivation.
- Phase 2 (40 mins): Sequential baseline implementation, warm-up profiling, and timing verification.
- Phase 3 (45 mins): Multi-threaded OpenMP scaling, Amdahl curve derivation, and reality gap mapping.
- Phase 4 (45 mins): Micro-architectural experiments (Triggering and resolving false sharing; scheduling evaluation).
- Phase 5 (30 mins): Technical analysis questions, verification checksum generation, and repository packaging.

2. Phase 1: Machine Probing & Workload Formulation (20 mins)

To guarantee integrity, your workload size N is tied to your Student Identification Number. Calculate your unique upper bound N before writing any code:

UNIQUE INPUT WORKLOAD FORMULA: $N = 10,000,000 + ([\text{Last 4 Digits of Student ID}] * 1,000)$
 Example: If Student ID is 20240984, Last 4 Digits = 0984. $N = 10,000,000 + (984 * 1,000) = 10,984,000$.

Step 1.1: Capture Hardware Specification. Open your terminal and pipe your hardware configuration into a raw text file named `hw_info.txt`. Include this file in your final submission:

```
# Linux Systems:
lscpu > hw_info.txt

# macOS (Darwin):
sysctl -a | grep -E "machdep.cpu|hw.ncpu|hw.l" > hw_info.txt

# Windows (PowerShell):
Get-CimInstance Win32_Processor | Select-Object Name, NumberOfCores,
NumberOfLogicalProcessors | Format-List > hw_info.txt
```

Step 1.2: Computational Kernel — The Collatz Stopping Time. For each integer i in $[1, N]$, calculate its stopping time (the number of steps required to reach 1 under the $3x + 1$ rules). Track the maximum stopping time observed across the entire range.

```
// Collatz Stopping Time Kernel
static inline uint32_t collatz_steps(uint64_t n) {
    uint32_t steps = 0;
    while (n > 1) {
        if ((n & 1) == 0) n >>= 1;
        else n = 3 * n + 1;
        steps++;
    }
    return steps;
}
```

3. Phase 2: Sequential Baseline & Precision Timing (40 mins)

Step 2.1: Implement `collatz_seq.c`. Your sequential program must iterate through $i = 1$ to N , calculate `collatz_steps(i)`, find the maximum steps, and sum all stopping times modulo 1,000,000,007 (to serve as an anti-cheat checksum).

Step 2.2: Benchmarking Rules & Warmup Invalidation. High-performance computing requires careful measurement:

- **Warmup Rule:** Discard Run 1 (Cold cache / OS page allocation penalty).
- **Timing Rule:** Execute Run 2 and Run 3 under identical machine state. Compute $T_{seq} = (Run\ 2 + Run\ 3) / 2.0$.
- **Isolation:** Close web browsers, streaming audio, and background IDEs during benchmarks to avoid core migration.
- **Clock Precision:** Use `omp_get_wtime()` for sub-microsecond wall-clock timing precision. Do NOT use `clock()`.

```
# Compile with standard optimization flags (-O2)
gcc -O2 -fopenmp collatz_seq.c -o collatz_seq
./collatz_seq
```

4. Phase 3: Multi-Thread Scaling & Empirical Amdahl Fitting (45 mins)

Parallelize your sequential loop using OpenMP (`#pragma omp parallel for`). You will evaluate the parallel runtime T_k across thread counts k in $\{1, 2, 4, 8, 16$ (if supported) $\}$.

Analytical Derivation of Parallel Fraction (p)

Using your empirical speedup at $k = 2$ threads ($S_{emp}(2) = T_{seq} / T_2$), solve algebraically for the parallel fraction p :

DERIVATION OF p FROM DUAL-CORE SPEEDUP: $S(2) = 1 / [(1 - p) + (p / 2)]$
 $1 / S(2) = 1 - p + 0.5p = 1 - 0.5p$
 $0.5p = 1 - (1 / S(2))$
 $p = 2 * [1 - (1 / S_{emp}(2))]$

Once p is derived, calculate the theoretical speedup $S_{theo}(k)$ for all other k values using Amdahl's Law, and calculate the Reality Gap: $\Delta(k) = S_{theo}(k) - S_{emp}(k)$.

Empirical Measurement Log: Record your raw benchmarks in Table 1 below:

Threads (k)	Run 1 (Cold)	Run 2 (s)	Run 3 (s)	Avg T_k (s)	$S_{emp}(k)$	$S_{theo}(k)$	$\Delta(k)$
k = 1 (T_1)	3.073249	3.064442	3.079592	---	---	---	---
k = 2	1.731127	1.736938	1.841905	---	---	---	---
k = 4	1.002855	0.986833	1.036696	---	---	---	---
k = 8	0.782513	0.733202	0.760734	---	---	---	---
k = 16 (or max)	---	---	---	---	---	---	---

5. Phase 4: Hardware Penalties & Micro-Architectural Experiments (45 mins)

Amdahl's model treats all parallel work as linear and uniform. In this phase, you will intentionally expose your code to two real-world hardware failure modes.

Experiment A: The False Sharing Penalty (Cache Invalidation)

Task: In addition to finding the maximum sequence length, count how many numbers in [1, N] require more than 100 stopping steps.

Implementation 1 (Naive Array - Triggers False Sharing): Declare an array `int hit_count[MAX_THREADS]` outside the parallel region. Inside the loop, each thread executes: `hit_count[omp_get_thread_num()]++`.

Implementation 2 (Mitigated - OpenMP Reduction or Padded Struct): Implement either an OpenMP reduction variable (`#pragma omp parallel for reduction(+:total_hits)`) or a padded structure that forces each thread's counter onto a separate 64-byte cache line (`struct Padded { int count; char pad[60]; };`).

Execute both implementations using your CPU's maximum physical thread count. Record the results below:

Implementation Variant	Thread Count	Execution Time (s)	Effective Throughput (iter/sec)	Speedup Penalty Ratio
Variant 1: Naive hits[tid]++ (False Sharing)	Max Physical: <u>4</u>	--- 1.689631	--- 7830705.75	---
Variant 2: Reduction / Cache-Padded	Max Physical: <u>4</u>	--- 0.976048	--- 13555687.69	---

Experiment B: Loop Scheduling & Workload Imbalance

Collatz sequence trajectories exhibit extreme non-uniformity: integer 26 terminates in 10 steps, whereas integer 27 requires 111 steps. Evaluate how OpenMP scheduling clauses impact thread tail latency at your maximum thread count:

OpenMP Scheduling Clause	Chunk Size	Execution Time (s)	Observed Behavior & CPU Load Distribution
<code>schedule(static)</code>	Default (N / k)	--- 0.981402	Baseline static partitioning across threads.
<code>schedule(static, 1000)</code>	1,000	--- 0.998483	Round-robin chunk distribution.
<code>schedule(dynamic, 100)</code>	100	--- 0.973163	Fine-grained dynamic work queue.
<code>schedule(dynamic, 10000)</code>	10,000	--- 0.963055	Coarse-grained dynamic work queue.
<code>schedule(guided)</code>	Exponential decay	--- 0.968063	Adaptive scheduling (starts large, reduces size).

6. Phase 5: Technical Analysis & Defense Questions (30 mins)

Answer the following questions concisely based on your empirical benchmark outputs:

Q1: Micro-Architectural Root Cause of False Sharing

Explain why Variant 1 in Experiment A caused massive execution degradation. Specifically reference your CPU's L1 cache line size (in bytes), the MESI/MOESI cache protocol, and bus invalidation traffic.

Variant 1 suffers from false sharing because the per-thread counters are stored in a normal contiguous integer array. Since an integer is much smaller than the CPU cache line, counters belonging to different threads can reside on the same 64-byte cache line. When one thread updates its counter, the cache coherence protocol must invalidate or update other cores' copies of that cache line. Under the MESI/MOESI coherence mechanism, repeated writes therefore generate cache-line ownership transfers and invalidation traffic between cores. The threads are logically updating independent variables, but the hardware treats them as parts of the same cache line. This creates significant coherence traffic and reduces performance. The cache-padded version separates the counters so that different threads operate on different cache lines, reducing this contention.

Q2: Hyperthreading (SMT) Saturation & Physical Core Ceilings

Did your speedup continue to scale linearly when thread count k increased from your physical core count to your logical thread count? Why or why not? What hardware execution resources are shared between SMT threads?

Q3: Empirical vs. Theoretical Amdahl Discrepancy

State your derived parallel fraction p (from $k=2$). Why did the theoretical curve $S_{\text{theo}}(k)$ diverge from your empirical $S_{\text{emp}}(k)$ as k increased to 8 or 16? Identify two real physical factors omitted by Amdahl's equation.

Q4: The Scheduling Trade-Off Dilemma

Compare dynamic(100) versus dynamic(10000) and static scheduling. At what chunk size did lock-queue contention outweigh the benefits of workload balancing on your machine?

7. Phase 6: Deliverables Checklist & Packaging Instructions (15 mins)

Package all lab assets into a compressed archive named `lab1_[StudentID].tar.gz` or push to a private Git repository with instructor access:

- **1. Hardware Specification:** `hw_info.txt` — Verbatim system output from `lscpu`, `sysctl`, or PowerShell.
- **2. Source Code:** `collatz.c` / `collatz.cpp` — Full C/C++ source code with compilation instructions in header comments.
- **3. Benchmark Dataset:** `results.csv` — Raw machine timing logs matching Table 1, Table 2, and Table 3.
- **4. Scalability Graph:** `speedup_plot.png` — High-resolution graph plotting k vs. $S_{\text{emp}}(k)$, $S_{\text{theo}}(k)$, and Linear Ideal.
- **5. Written Defense:** `analysis.pdf` or written responses on this worksheet — Completed technical questions Q1 to Q4.

```
# How to create your submission archive:
mkdir lab1_20240984
cp hw_info.txt collatz.c results.csv speedup_plot.png lab1_20240984/
tar -czvf lab1_20240984.tar.gz lab1_20240984/
# Verify integrity before submission:
tar -tvf lab1_20240984.tar.gz
```

8. Evaluation & Grading Rubric (100 Points Total)

Assessment Category	Target Points	Proficient Criteria (Full Marks)	Deficiency Penalties (-10 to -25 pts)
Compilation & Correctness	20 pts	Clean compilation with -O2 -fopenmp. Verification checksum correctly matches the student ID workload N.	Compiler warnings, incorrect reduction, or checksum mismatch (-20 pts).
Empirical Timing Quality	30 pts	Realistic wall-clock benchmark data with warmup runs discarded; believable variance across runs; hw_info matches results.	Synthetic/simulated numbers, zero run variance, missing hardware specs (-30 pts).
Amdahl Fitting & Plotting	25 pts	Correct algebraic derivation of p from $S(2)$; accurate projection of $S_{theo}(k)$; clear plot showing the Reality Gap.	Incorrect derivation of p, missing speedup plot, or non-convergent curves (-15 pts).
Hardware Penalty Analysis	25 pts	Complete implementation of false sharing experiment; correct technical analysis of MESI protocol and scheduling trade-offs.	Superficial or incorrect explanation of cache invalidation; failure to mitigate false sharing (-15 pts).